



NITK Surathkal

TEAM NO RISK V

Detailed Architecture Proposal

SentinelSoC

ED25519 and DIFT based SoC for secure IoT applications

Mentors:

- Dr. Sumam David, Professor HAG, ECE dept., NITK
- Dr. Ramesh Kini, Professor and Head, ECE dept., NITK
- Mr. Udaya Kumar Bobbili, Senior Manager (R&D), Synopsis

Team:

- Abhinav S Rao
- Arya Shedbal
- Ishita Shankar
- Joseph Varghese
- Kushal Gowda C R
- Maanya Golash
- Shridhar Rajashekhara Bhat
- Vibhu Kav Bhat

Table of Contents

1. Introduction.....	3
1.1 Chip Overview:.....	3
1.2 Design Objectives:.....	4
1.3 Threat Model:.....	4
2. Security Architecture.....	4
2.1 Root of Trust:.....	4
2.2 Privilege Modes:.....	4
3. SoC Architecture.....	5
3.1 Top Level Block Diagram:.....	5
3.2 RISC-V CPU Subsystem:.....	6
3.3 ED25519 Cryptographic Subsystem:.....	12
3.3.1 SHA-512:.....	15
3.4 Dynamic Information Flow Tracking.....	18
3.5 Memory Subsystem:.....	25
4. References.....	28

1. Introduction

1.1 Chip Overview:

Modern IoT and embedded devices remain critically vulnerable to low-level hardware attacks, a threat that carries severe consequences in high-stakes domains such as healthcare, defence, and industrial control systems. At this level of access, a miscreant can extract sensitive data, corrupt computational outputs, and trigger cascading safety failures that are difficult to detect and nearly impossible to reverse.

Our chip directly addresses this exposed attack surface through two complementary hardware-level mechanisms: Boot Authentication, which ensures only cryptographically verified firmware is ever executed, and Dynamic Information Flow Tracking (DIFT), which monitors data movement across the chip in real time to detect and contain malicious activity as it happens. Together, these features close the backdoor that conventional software security leaves open, without compromising on performance.

SentinelSoC is built on the standard RV32IMC RISC-V architecture and integrates a standard peripheral suite and a JTAG Debug Interface. A full summary of the chip's modules and target specifications is provided below:

Module	Target Value
Processor Core	RV32IMC - 3 Stage pipeline - M/U Privilege Modes
Firmware auth	SHA-512 (hash) + ED25519 (Signature)
DIFT	Dynamic Information Flow Tracking, 1-bit taint per word, hardware propagation
Bus Protocols	OBI + APB + OBI to APB bridge
On-Chip SRAM	8kb + 1tag bit(D-sram), 4kb i-sram
Boot Rom	1kb
External Flash	QSPI
Peripherals	GPIO, UART, Timer, SPI
Interrupt Controller	CLINT - timer&SW, PLIC - external
OTP	Key storage for ED25519 public key hash (256 bit)
Debug	RISC-V Debug Module (JTAG DTM)
Target Applications	Secure IoT devices, Hardware Root-of-Trust, Firmware Authentication

1.2 Design Objectives:

- Track data flow via tags
- Reduce accelerator, tag memory overhead
- Effectively prevent external threats through 2 security layers (pre boot, runtime)
- Pipeline execution with eliminating hazards and unnecessary stalls
- Provide memory protection via PMP blocks
- Additional CSRs to define security policy
- SHA Module for accurate hashing for comparison
- Signature verification of firmware
- Modular design to allow correction and upgrades
- Extensive modelling to prevent edge cases

1.3 Threat Model:

- Untrusted firmware operating SoC
- Buffer Overflow attack
- Memory corruption
- Illegal sensitive register writes or Root of Trust

2. Security Architecture

2.1 Root of Trust:

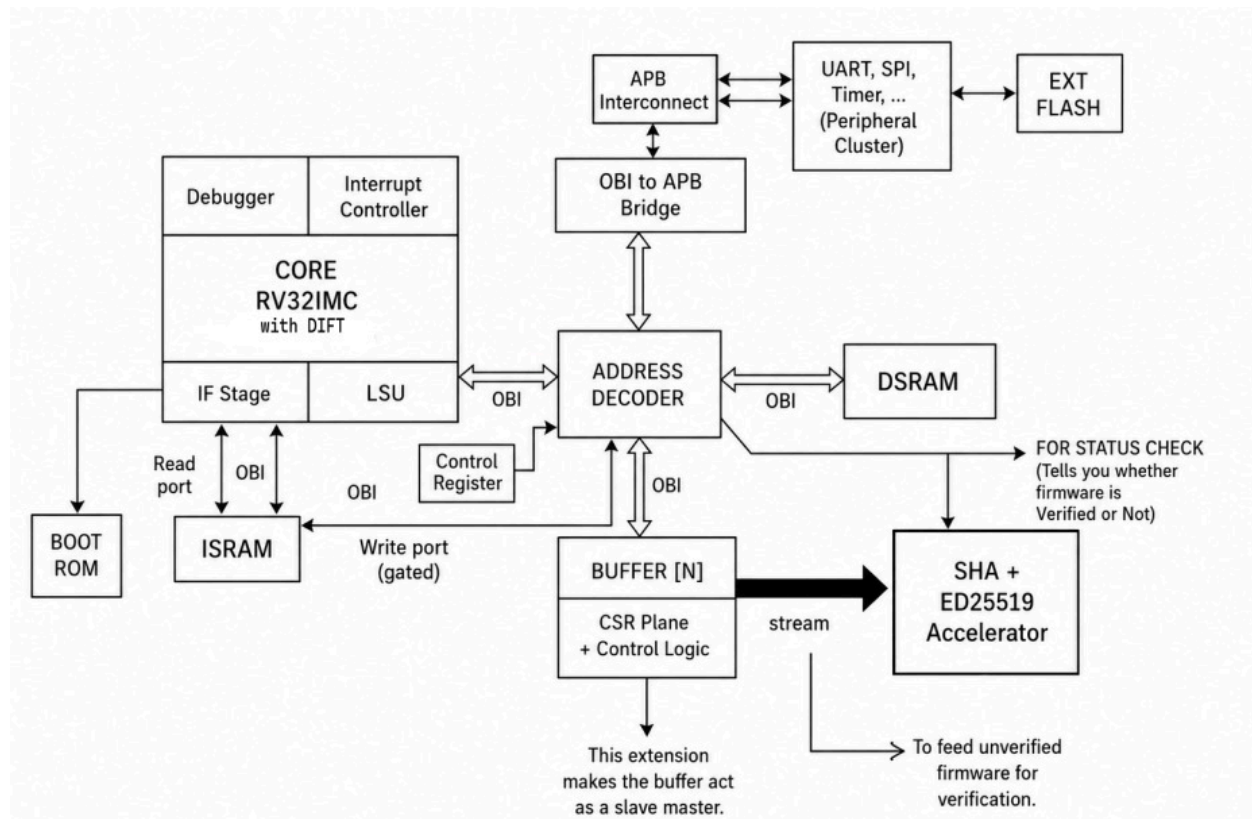
- One Time Programmable Memory (OTP)
- BootROM
- ED25519

2.2 Privilege Modes:

- Machine Mode (M)
- User Mode (U)

3. SoC Architecture

3.1 Top Level Block Diagram:

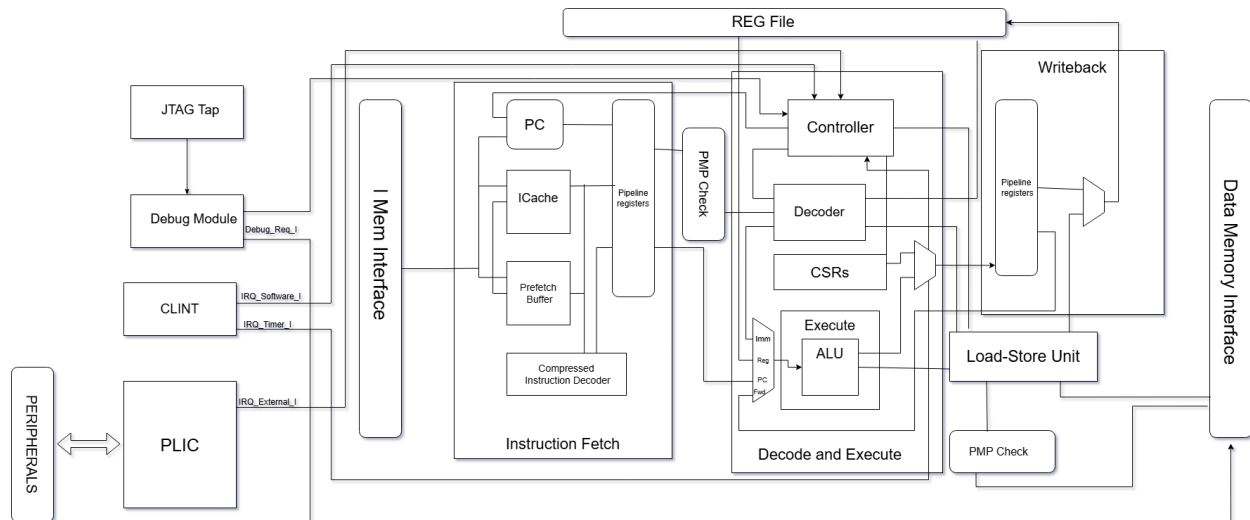


3.2 RISC-V CPU Subsystem:

Introduction:

The processor subsystem is the computational core of the SoC. It is built around a 3-stage in-order pipelined RV32IMC processor implementing the RISC-V Base Integer (I), Integer Multiply/Divide (M) and Compressed Instruction (C) extensions. Beyond the core pipeline itself,

the subsystem includes a JTAG-based debug interface made up of a JTAG TAP and an external Debug Module, The CPU and Debug Module are two bus masters, arbitrated by a 2-to-1 OBI arbiter before the address decoder, along with two interrupt management peripherals: the Core Local Interruptor (CLINT) and the Platform-Level Interrupt Controller (PLIC), both of which feed the core through its dedicated interrupt input pins.



Core Architecture:

The processor core is organized as a 3-stage pipeline covering Instruction Fetch (IF), Instruction Decode and Execute (ID/EX), and Writeback (WB).

The core is compliant with the RISC-V Privileged Architecture specification v1.11 and supports Machine (M) and User (U) privilege modes.

Pipeline Stages:

- **Instruction Fetch (IF):** The program counter drives the instruction cache (ICache) and prefetch buffer to supply a continuous instruction stream to the pipeline. A PMP check is applied at the fetch boundary to enforce instruction-fetch access permissions.
- **Instruction Decode and Execute (ID/EX):** This stage brings together the instruction decoder, register file read ports, ALU, multiplier/divider, Load-Store Unit (LSU), CSR file, and the Controller FSM. The Controller is responsible for resolving data hazards, managing pipeline stalls and flushes, and handling exceptions, interrupts, and debug events. Branch and jump targets are resolved here, and a taken branch or jump causes a 1-cycle flush of the instruction sitting in the IF stage.
- **Writeback (WB):** this stage holds ALU results or load data from the LSU until they can be committed to the register file. Results can be forwarded from WB back to ID/EX to

resolve data hazards without stalling. A PMP check is applied on the data memory path at this stage before completing store operations.

ISA Stages:

The core implements the following RISC-V ISA extensions:

Extension	Name	Description
I	Base Integer	32-bit base integer ISA covering arithmetic, logic, shift, load/store, and control-flow instructions
M	Integer Multiply/Divide	MUL, MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU. The multiplier is configurable as slow (multi-cycle), fast, or single-cycle.
C	Compressed Instructions	16-bit compressed encodings for common instructions, reducing code footprint by roughly 25 to 30 percent. Since all PCs are 16-bit aligned, instruction-address-misaligned exceptions are eliminated.
ZiCSR	CSR Instructions	CSRRW, CSRRS, CSRRC and their immediate variants for CSR access

Hazard Management and Forwarding:

Data hazards from producer-consumer instruction sequences are handled through a combination of result forwarding and pipeline stalls. Results from the WB stage are forwarded back to the ID/EX stage operand inputs to avoid unnecessary stalls wherever possible. Load-use hazards, where the result of a load is consumed by the very next instruction, require a mandatory 1-cycle stall since the load data is not available until the memory response returns.

Control hazards from branches and jumps are resolved in the ID/EX stage. A taken branch or unconditional jump flushes the instruction in the IF stage, incurring a 1-cycle penalty. The Controller FSM also handles multi-cycle instructions such as multiply, divide, and unaligned loads/stores by stalling the pipeline until completion.

Physical Memory Protection (PMP) :

The core implements PMP in accordance with the RISC-V Privileged Specification v1.11. PMP entries define address ranges along with associated read (R), write (W), and execute (X) permissions. Checks are enforced at two points in the pipeline: at the IF stage boundary for instruction fetches, and at the LSU boundary in the WB stage for data memory accesses. A permission violation raises a synchronous exception, saves the faulting PC to mepc, and transfers execution to the machine-mode trap handler via mtvec.

JTAG Based Debug Interface :

Run-control debug support is provided through a chain of three components: a JTAG TAP controller, a Debug Module (DM) compliant with the RISC-V External Debug Support specification v0.13.2, and the debug logic built into the core itself. The JTAG TAP and Debug Module are instantiated as separate blocks outside the core and communicate with it through the dedicated debug_req_i input pin.

JTAG Tap Controller :

The JTAG TAP implements the IEEE 1149.1 standard four-signal interface (TCK, TMS, TDI, TDO). It presents the RISC-V Debug Transport Module (DTM) as a JTAG data register, allowing a host debugger such as OpenOCD to send debug commands over the standard JTAG interface. The DTM translates JTAG scan operations into transactions on the Debug Module Interface (DMI), which carries abstract register-mapped commands to the Debug Module.

Debug Module :

The Debug Module implements the RISC-V External Debug Support specification. It communicates halt requests to the core via the debug_req_i signal. Once the core enters Debug Mode, the Debug Module allows the host to perform the following operations:

- Halt and resume the hart
- Read and write general-purpose registers and CSRs via abstract commands
- Perform single-step execution through the dcsr.step bit
- Access system memory using the Program Buffer mechanism
- Assert a system reset via ndm_reset

Core Debug Support :

The core implements four debug CSRs as specified in the RISC-V Debug Specification v0.13.2: dcsr (Debug Control and Status), dpc (Debug PC), dscratch0, and dscratch1. When debug_req_i is asserted, the core waits for the instruction currently in the ID/EX stage to complete, then transitions into Debug Mode and freezes the pipeline, redirecting execution to the debug ROM. While in Debug Mode, all interrupts including NMI are suppressed regardless of the state of mstatus.MIE. If the DbgTriggerEn parameter is enabled, one hardware breakpoint trigger supporting instruction address matching is also available.

Interrupt Architecture :

The core exposes three dedicated level-sensitive interrupt input pins: irq_software_i, irq_timer_i, and irq_external_i. These correspond to machine software, timer, and external interrupts respectively as defined in the RISC-V Privileged Specification. The pins are driven by two SoC-level peripheral blocks, the CLINT and the PLIC, which are described in the following subsections.

CLINT - Core Local Interruptor :

The CLINT is a memory-mapped peripheral responsible for generating machine-mode timer and software interrupts. It drives the core's irq_timer_i and irq_software_i pins directly.

- **Machine Timer Interrupt (irq_timer_i):** The CLINT contains a 64-bit real-time counter (mtime) and a per-hart compare register (mtimecmp). A timer interrupt is asserted when mtime is greater than or equal to mtimecmp and stays level-high until software updates mtimecmp to a future value. This is the standard mechanism used for OS scheduling ticks and software timeouts.
- **Machine Software Interrupt (irq_software_i):** A software interrupt is triggered by writing to the memory-mapped msip register in the CLINT. This is typically used for self-interruption or for cross-hart signaling in multi-core configurations.

PLIC - Platform Level Interrupt Controller :

The PLIC is a memory-mapped peripheral that collects interrupt requests from external SoC peripherals and presents a single aggregated interrupt to the core's irq_external_i pin. It

implements the RISC-V PLIC specification with priority-based arbitration and per-source enable masking. The general flow of operation is as follows:

- Each interrupt source is assigned a programmable priority level by software.
- A global priority threshold register allows the core to mask all sources at or below the set threshold.
- The PLIC asserts `irq_external_i` when any enabled source with priority above the threshold is pending.
- The core reads the PLIC claim register to identify and atomically acknowledge the highest-priority pending source.
- After the interrupt has been serviced, the core writes the source ID back to the PLIC completion register to re-arm it.

Peripherals :

UART - serial communication for debug output and host communication; connects to PLIC as an interrupt source

GPIO - general-purpose I/O for board-level control and signaling; can also be wired to PLIC for pin-change interrupts

SPI - high-speed synchronous interface for flash memory or external sensors

I2C - low-speed two-wire bus for configuration registers, EEPROMs, or sensor peripherals

Timer/PWM - additional timers beyond the CLINT's mtime, used for PWM generation or application-level timeouts

Interrupt Handling in the Core :

Interrupt handling in the core follows the RISC-V Privileged Specification v1.11. The core operates exclusively in vectored interrupt mode: the `MODE` field of the `mtvec` CSR is hardwired to 01 (vectored) and is read-only. Synchronous exceptions always jump to the `mtvec` base address. Maskable interrupts are dispatched to `mtvec` base + 4 x `interrupt_ID`, giving each interrupt type its own dedicated handler entry point. The `mtvec` base address must be aligned to a 256-byte boundary.

When a maskable interrupt is asserted, the core first checks the global interrupt enable bit (`mstatus.MIE`) and the per-source enable bit in the `mie` CSR. If both are set, the core waits for the current instruction in the ID/EX stage to retire, saves the return address to `mepc`, records the interrupt cause in `mcause`, copies `mstatus.MIE` into `mstatus.MPIE`, clears `MIE`, and jumps to the vectored handler address. All interrupt lines are level-sensitive, so the interrupt source is

expected to deassert its line once it receives acknowledgment from the handler. On execution of MRET, the core restores mstatus.MPIE back into mstatus.MIE and resumes execution from mepc.

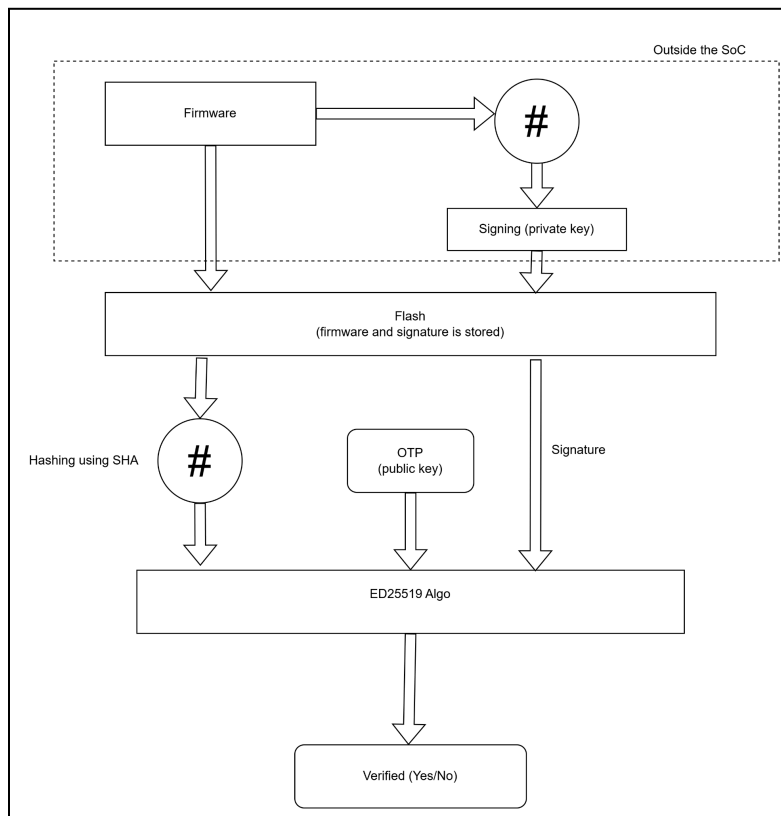
3.3 ED25519 Cryptographic Subsystem:

Ed25519 is a modern elliptic curve signature scheme that prioritizes speed, compactness, and side-channel resistance without compromising cryptographic strength. It operates on a complete curve formula with no conditional branching during point addition, enforcing constant-time execution by construction and eliminating a broad class of timing attacks. Nonce generation is fully deterministic, removing any dependence on physical entropy sources. These properties make it a natural fit for hardware acceleration under strict area, power, and timing constraints, which is precisely the environment SentinelSoC operates in.

Firmware signing occurs offline, outside the SoC, using a private key held by the firmware author. The signing process produces a 64-byte Ed25519 signature which is stored in flash alongside the firmware image, ready to be verified at boot time.

At boot, the CPU bootloader reads firmware and its accompanying signature (R, S) word by word from external flash via the QSPI controller. The CPU writes these words to the firmware buffer via OBI. The buffer streams the firmware to the SHA+ED25519 subsystem. The trusted public key is retrieved directly from OTP - hardwired into the subsystem with no CPU involvement. The SHA engine hashes the firmware, and an internal glue FSM reads the digest and loads it into the ED25519 register file alongside S, R, and pubkey before triggering verification.

High Level Functional Architecture:



ED Signing pre - SOC

```
EdDSA_sign(msg, privKey) --> { R, s }
```

1. Calculate **pubKey** = **privKey** * **G**
2. Deterministically generate a secret integer **r** = hash(hash(**privKey**) + **msg**) mod **q** (this is a bit simplified)
3. Calculate the public key point behind **r** by multiplying it by the curve generator: **R** = **r** * **G**
4. Calculate **h** = hash(**R** + **pubKey** + **msg**) mod **q**
5. Calculate **s** = (**r** + **h** * **privKey**) mod **q**
6. Return the **signature** { **R**, **s** }

ED Verifier: In SOC Accelerator

```
EdDSA_signature_verify(msg, pubKey, signature { R, s } ) --> valid /  
invalid
```

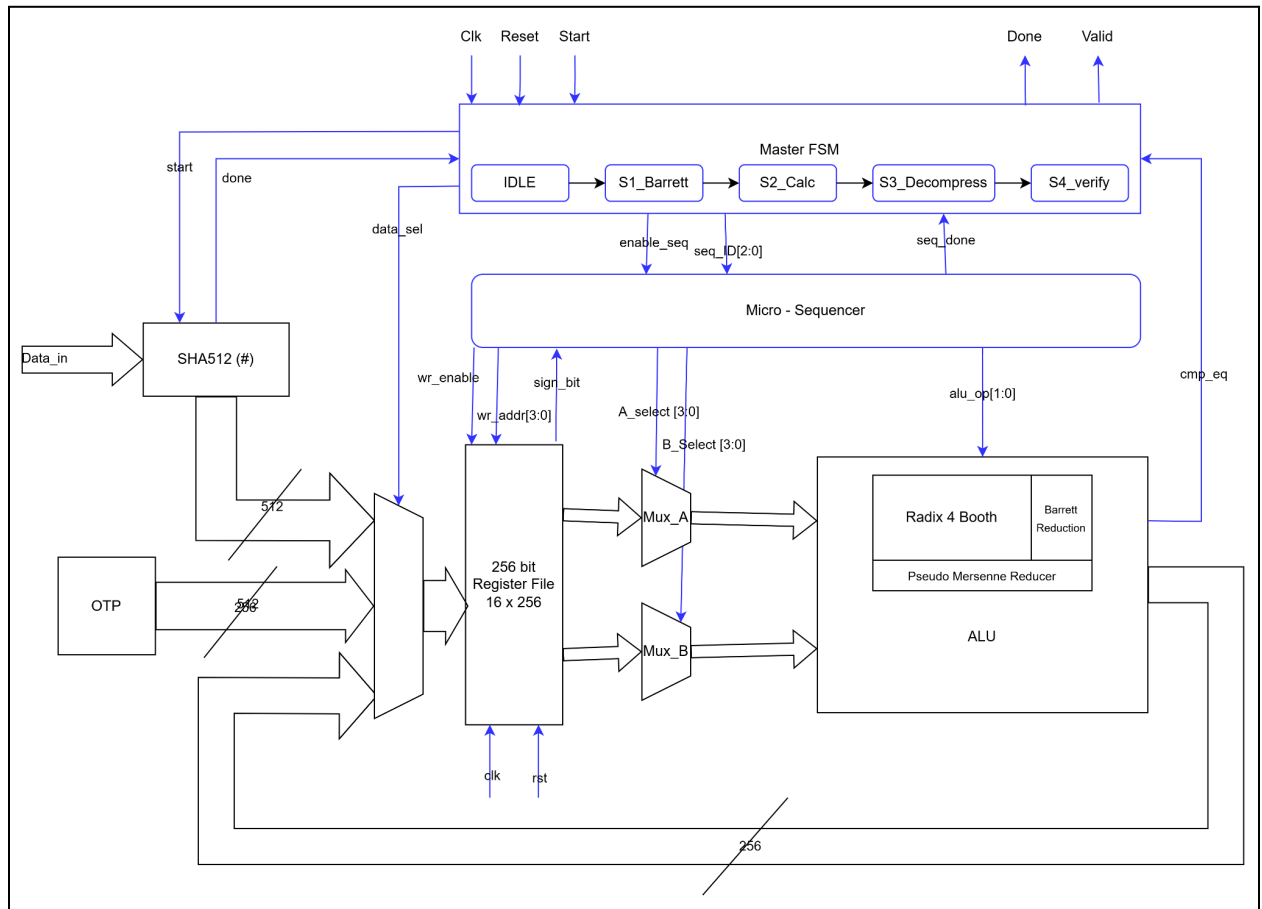
1. Calculate $h = \text{hash}(R + \text{pubKey} + \text{msg}) \bmod q$
2. Calculate $P1 = s * G$
3. Calculate $P2 = R + h * \text{pubKey}$
4. Return $P1 == P2$

Implementing these 4 steps requires the following blocks:

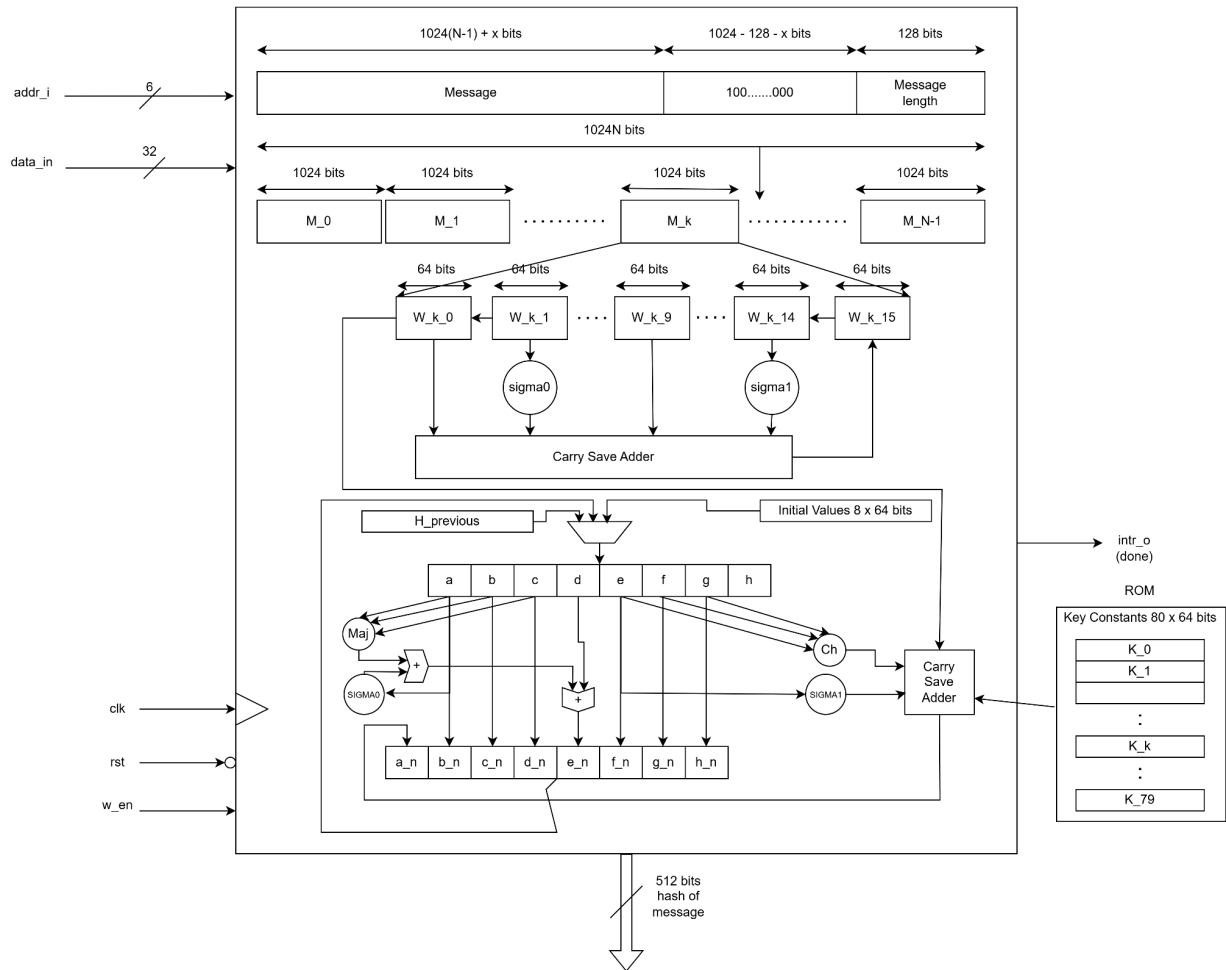
- SHA-512 Hashing Algorithm
- Radix 4 Booth Multiplier
- Barrett Reduction
- Pseudo Mersenne Reduction
- Point Double and Point Add

Despite the apparent complexity, they all use the same fundamental Booth Multiplier + Pseudo Mersenne or Barrett reducer based on different sequences. This is implemented using the microsequencers, controlled by the main Master FSM.

A more detailed microarchitecture is given below:



3.3.1 SHA-512:



The SHA-512 module is one of the math-heavy blocks within the ED25519 accelerator. It implements the SHA-512 algorithm as defined by NIST. It works by accepting an input of any bit length and returning a hash of exactly 512 bits.

- The formatted input is split into blocks of 1024 bits. Each 1024 bit block is used to produce 80 “Words” of 64 bits each.
- 1 word is processed in a single cycle with eight 64-bit registers ‘a’ to ‘h’ that are originally loaded with the initial values in case of the first block (or) the Hash of the previous block for all subsequent blocks.
- In each round, the output is fed as the initial value to the next block of 1024 bits.
- In the case of the last block, the current output is added back to the output of the previous block which gives the final hash.
- It also contains eight 64bit registers ‘a’ to ‘h’ which are updated every cycle and eight 64bit registers H[0] to H[7] to store the result at each stage.
- The SHA-512 algorithm calls for 80 registers to store 64 bit “words” - one to operate on in each round.

- Our design reduces this into an array of only 16 registers of 64 bits and we dynamically calculate the next word (as shown in the datapath) and discard the current word after operation.
- The number 16 is chosen since the least recent word that $W(k)$ depends on is $W(k-16)$.

In each round, $W[0]$ is the current word, while $W[16]$ is newly calculated as

$$W[16] = \sigma_1(w[14]) + w[9] + \sigma_0(w[1]) + w[0]$$

Where σ_0 and σ_1 are functions defined as

$$\sigma_0(x) = \text{ROTR1}(x) \wedge \text{ROTR8}(x) \wedge \text{SHR7}(x)$$

$$\sigma_1(x) = \text{ROTR19}(x) \wedge \text{ROTR61}(x) \wedge \text{SHR6}(x)$$

Where ROTR_n and SHR_n stand for Rotate Right by n and Shift Right by n respectively and $\wedge$ stands for bitwise XOR.

The state transformations that follow in each of the 80 rounds are as follows:

$$a_n = t1 + t2$$

$$b_n = a$$

$$c_n = b$$

$$d_n = c$$

$$e_n = d + t1$$

$$f_n = e$$

$$g_n = f$$

$$h_n = g$$

Where $t1$ and $t2$ are given by

$$t1 = h + \Sigma_1(e) + \text{Ch}(e, f, g) + Kt + Wt$$

$$t2 = \Sigma_0(a) + \text{Maj}(a, b, c)$$

Where Σ_0 , Σ_1 , Maj and Ch are functions defined as

$$\text{Maj}(a, b, c) = \text{bitwise majority bit among } a, b, c$$

$$\text{Ch}(e, f, g) = (e \& f) \mid (e \& \sim g)$$

$$\Sigma_0(x) = \text{ROTR28}(x) \wedge \text{ROTR34}(x) \wedge \text{ROTR39}(x)$$

$$\Sigma_1(x) = \text{ROTR14}(x) \wedge \text{ROTR18}(x) \wedge \text{ROTR41}(x)$$

Where ROTR_n stands for Rotate Right by n and $\wedge$ stands for bitwise XOR.

Calculation of $W[16]$ and $t1$ is done using a Carry save adder as they are part of the critical path. We also let the next initial value $H = H + a..h$ infer eight 64 bit adders for the same reason.

The control path is implemented via an FSM having three states - IDLE, WORK and ACCUM. WORK state runs through the 80 rounds of the algorithm and ACCUM state performs the final addition in each block. The IDLE state indicates that the datapath is inactive and the module is ready to receive the input via the interface. The interface handles 2 things:

1. Reversing the little endian data to make it suitable for the big endian SHA block
2. Performing the formatting and scheduling the input data to arrive in blocks of 32 bits.

The data bus either carries the scheduled data or control signals based on the contents of the address bus. The `intr_o` signal tells the parent ED25519 block that hashing is complete.

addr_i value	data_i and data_o	Explanation
0 to 31	Valid input address: data_i carries input data	addr[4:1] gives input word number and addr_i[0] indicates lower or higher 32 bits of the input word
32	data_i carries control signals	Start and init_cmd signals: begin writing
33	data_o carries control signals	Done signal: begin reading
34 to 49	Valid output address: data_o carries output data	addr[4:1]-17 gives output word number and addr_i[0] indicates lower or higher 32 bits of the output word

Specs:

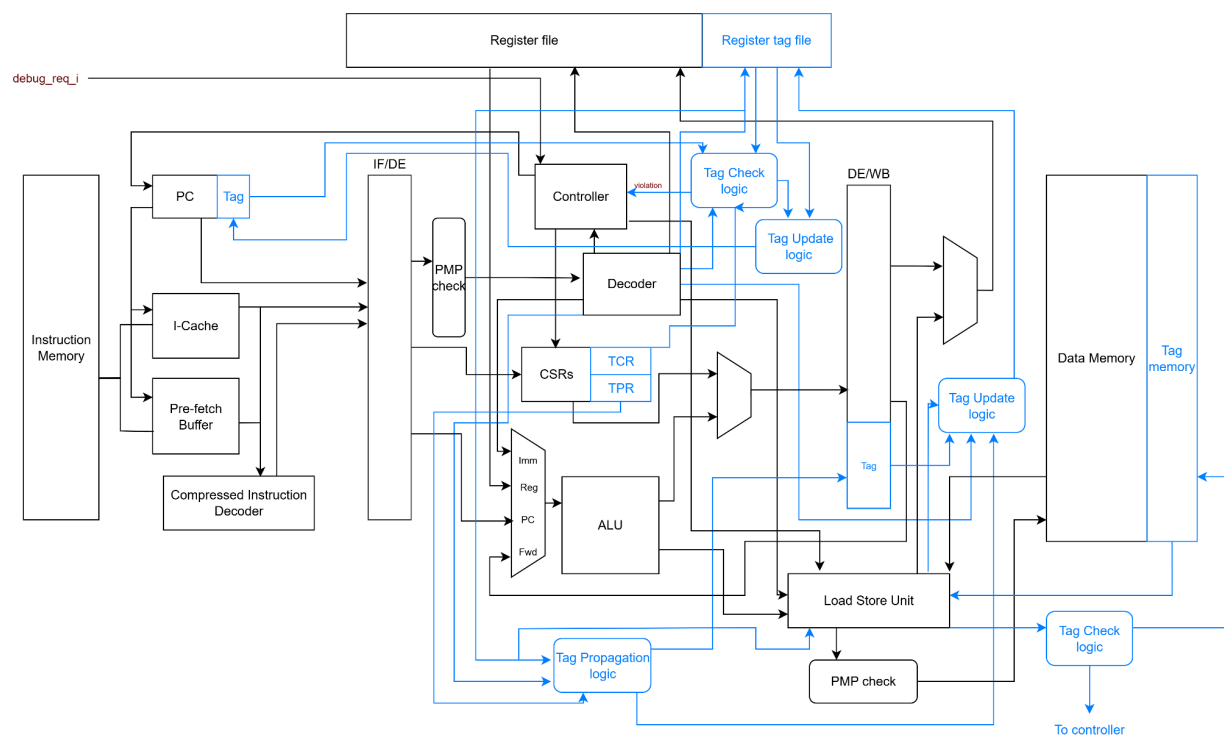
Memory (ROM)	800 Bytes
Adders	8
CSAs	1
Shift registers	3

3.4 Dynamic Information Flow Tracking

Dynamic Information Flow Tracking (DIFT) is a hardware-assisted security mechanism used to monitor and control how information propagates through a processor during execution, to protect the SoC from buffer overflow attacks and illegal writes. Each data element in the system is associated with a **tag bit** that indicates whether the data is trusted or untrusted. These tags propagate alongside data through the processor pipeline according to defined security policies.

The DIFT system implemented in this architecture integrates with the **3 stage pipelined RISC-V core** and introduces additional hardware modules to track, propagate, and enforce tag-based security policies as given below.

DIFT on 3 stage RISC-V Pipeline (Datapath)



Lines in blue indicate additional hardware and wires required to be added to the existing 3 stage pipelined RISC V core. It consists of 7 main units:

- **CSRs** (TPR : Tag propagation register, TCR : Tag check register)
- **Tag registers** (Parallel to General purpose registers, PC, DE/WB pipeline register)
- **Tag memory** (parallelly indexed with SRAM)
- **Tag Propagation logic** : Combinational logic circuitry to propagate tags through the pipeline and w.r.t. Instructions, according to a hardwired policy
- **Tag Check logic** : Combinational logic logic circuitry to check tags and prevent illegal branches and stores to the memory according to a hardwired policy
- **Tag update logic** : Combinational logic to update tags into tag registers in the writeback stage

Tag Flow:

Register Tag File → Tag Propagation Unit → Tag Check Unit → Tag Update Unit → Register Tag File

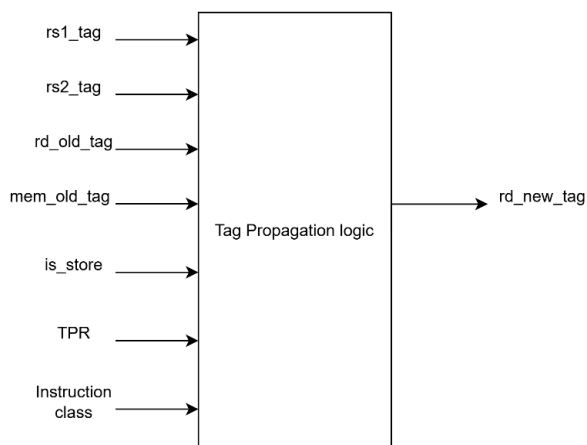
PC tag → Tag Check Unit → Tag Update Unit → PC tag

Tag value	Meaning
0	Trusted data
1	Tainted / Untrusted data

Tag Sources in the SoC:

- **External Data Sources** : Data entering through peripherals or memory interfaces marked as untrusted
- **Tag Memory** : Each memory location has an associated tag stored in a Tag Memory
- **Register Tag File** : stores the tags corresponding to general-purpose registers.
- **PC Tag** : maintains the tag associated with control-flow instructions.

TPR and Tag propagation unit Architecture-



The **TPR register** determines how tags propagate between operands and results.

Structure: (17-bit CSR) Contains propagation rules for 7 different instruction classes.

Each class has a **2-bit propagation mode**, while load/store operations also include 3 enable bits controlling tag participation.

Propagation modes (These rules define how operand tags generate the result tag) :

Mode	Operation
00	Preserve destination tag
01	AND propagation
10	OR propagation
11	Clear tag

The **Tag Propagation Unit** computes the tag for the instruction result.

Inputs:

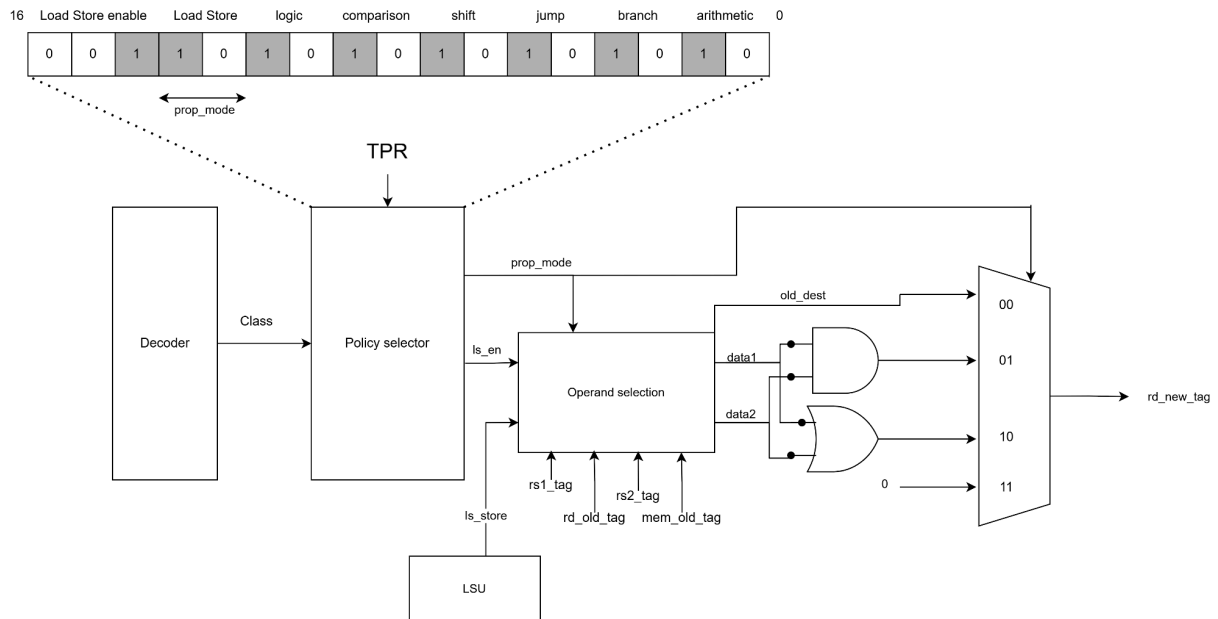
rs1_tag
rs2_tag
rd_old_tag
mem_old_tag
instruction class
propagation policy (TPR)

Operation:

1. Instruction class selects the propagation rule from TPR.
2. Operand tags are selected depending on instruction type.
3. The propagation rule computes the candidate result tag.

Output:

rd_tag_candidate / rd_new_tag



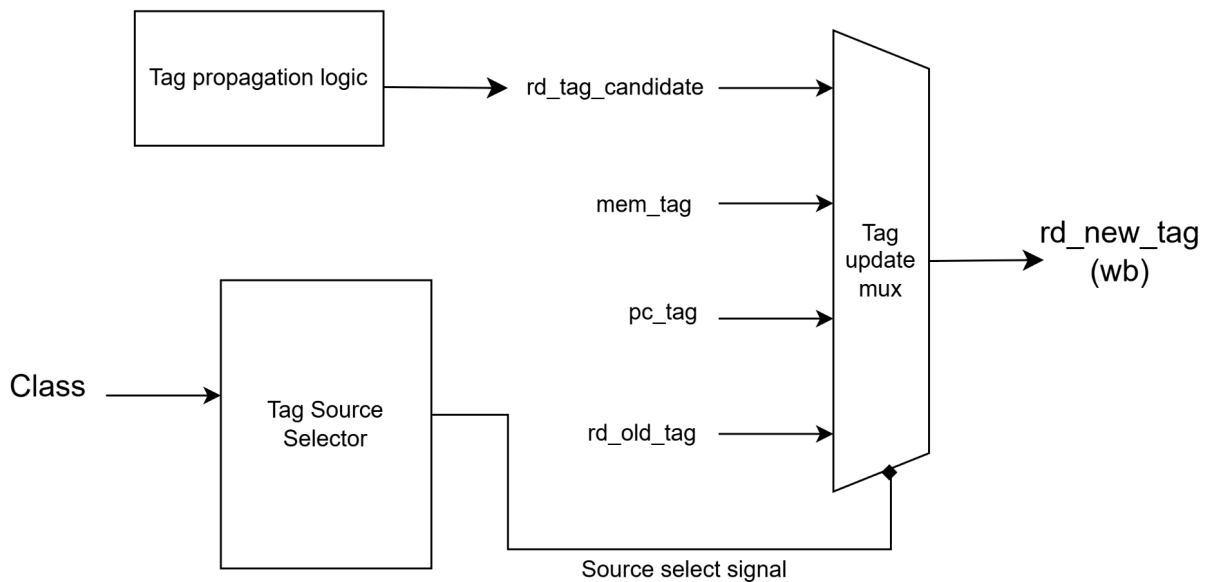
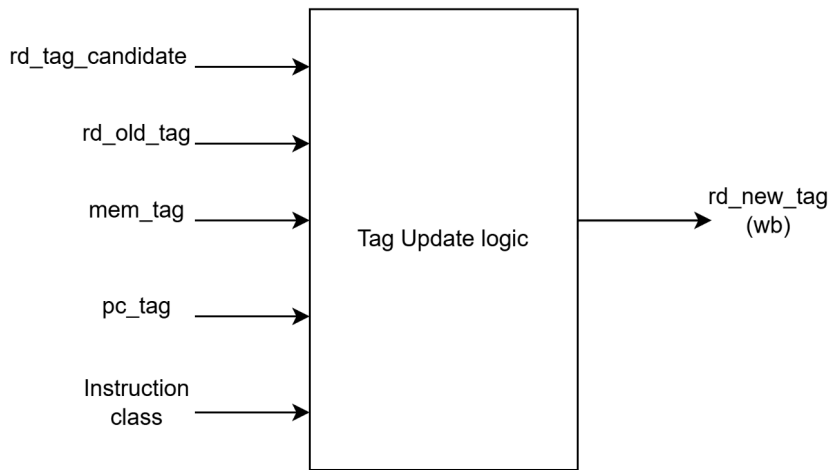
Tag Update Unit

The **Tag Update Unit** determines the tag written to the destination register.

Possible sources:

- propagated tag (**rd_tag_candidate**)
- memory tag (**mem_tag**) (from LSU)
- program counter tag (**pc_tag**)
- previous destination tag (**rd_old_tag**)

The correct tag source is selected based on instruction type and written back to the **Tag Register File** in the **writeback stage**.



Tag Check Register and Tag Check Unit Architecture:

The TCR (22 bits) defines security checks for each instruction class using a check mode field. All instruction classes have a 3 bit check mode field except LS type instructions (4 bit), Branch (2 bit) and Exec_check for PC tag (1 bit).

However for hardware simplicity, checkmode is taken as 4bits and homogenised by padding 0 for other instruction classes (except exec_check)

If an enabled check detects a tainted value where it is not allowed, a violation signal is generated.

The **Tag Check Unit** verifies that tag values comply with the policy defined in TCR.

Inputs:

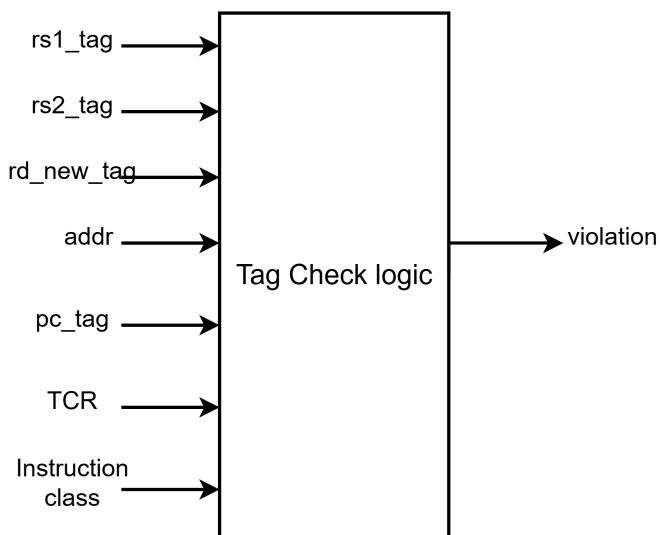
- rs1_tag
- rs2_tag
- rd_new_tag
- address tag
- pc_tag
- check policy (TCR)

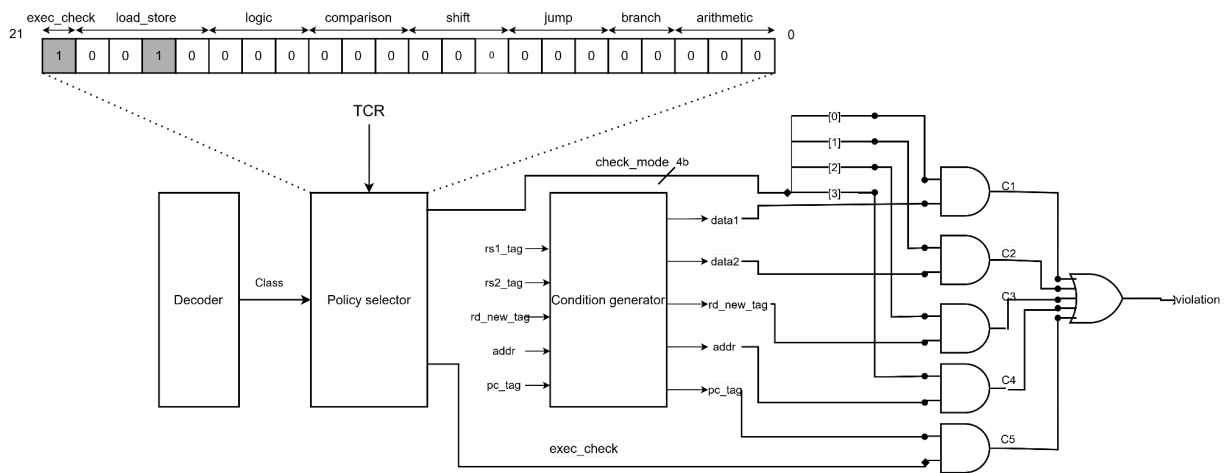
Operation:

1. Instruction class selects the relevant check policy.
2. Tag conditions are evaluated.
3. Enabled conditions are combined.

If any condition is violated: **violation = 1**

The signal is forwarded to the processor controller.





Violation Handling

When a policy violation occurs:

1. The **violation signal** is asserted.
2. The processor controller triggers an exception.
3. The pipeline is flushed.
4. Execution transfers to the trap handler.

3.5 Memory Subsystem:

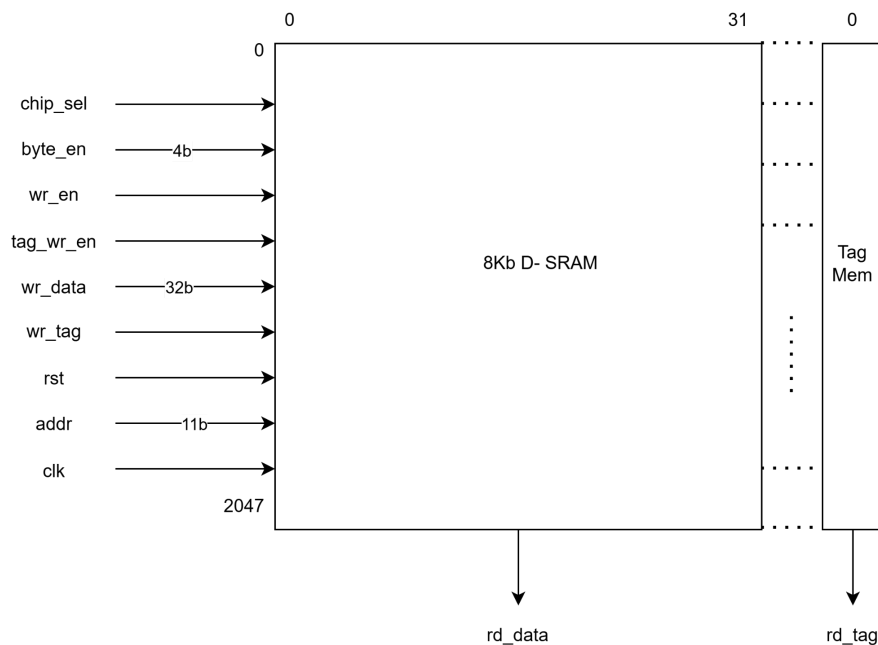
The architecture integrates the following memory blocks onto the SOC:

Feature	Specification	Security Contribution
BootROM	1KB	Root of Trust
OTP	256-bit	Permanent Secure Key Storage
D-SRAM	8KB + 1-bit Tag per word	Information Flow Tracking
I-SRAM	4KB	Fast, dedicated instruction space

Data SRAM (D-SRAM) with Integrated Tagging:

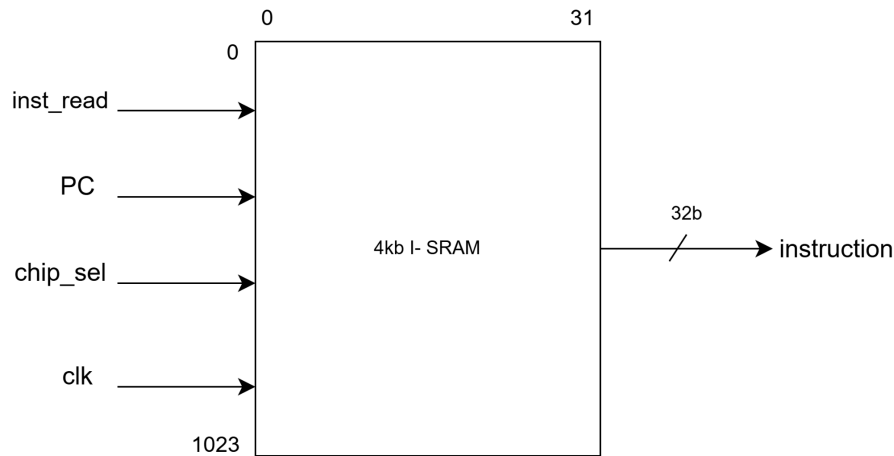
- **Capacity:** It is an on-chip 8KB D-SRAM with a parallelly indexed 1-bit array of 2048 entries to store tag-bits.
- **Architecture:** The tag memory and D-SRAM is word addressed and have the same address locations. Each memory access fetches the 32-bit word and its assigned 1-bit tag data together in a single fetch cycle.
- **Efficiency:** The design achieves advanced hardware data security tracking with only a 3.125% overhead over standard D-SRAM.

This memory design modification is made for the implementation of Dynamic Information Flow Tracking (DIFT).



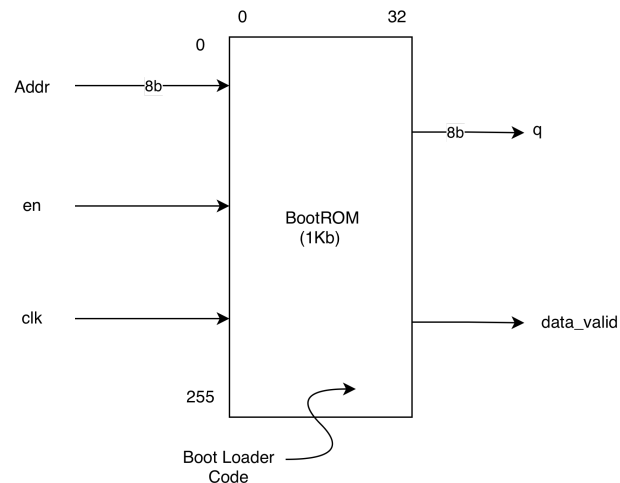
Instruction SRAM (I-SRAM):

- **Capacity:** A 4KB dedicated on-chip memory for instructions.
- **Design:** It contains CPU instructions that can be accessed by the program counter (PC). No tag-bits are assigned to the instructions, hence it does not have a parallel tag memory.
- The I-SRAM is implemented as a single-port synchronous SRAM macro.



BootROM:

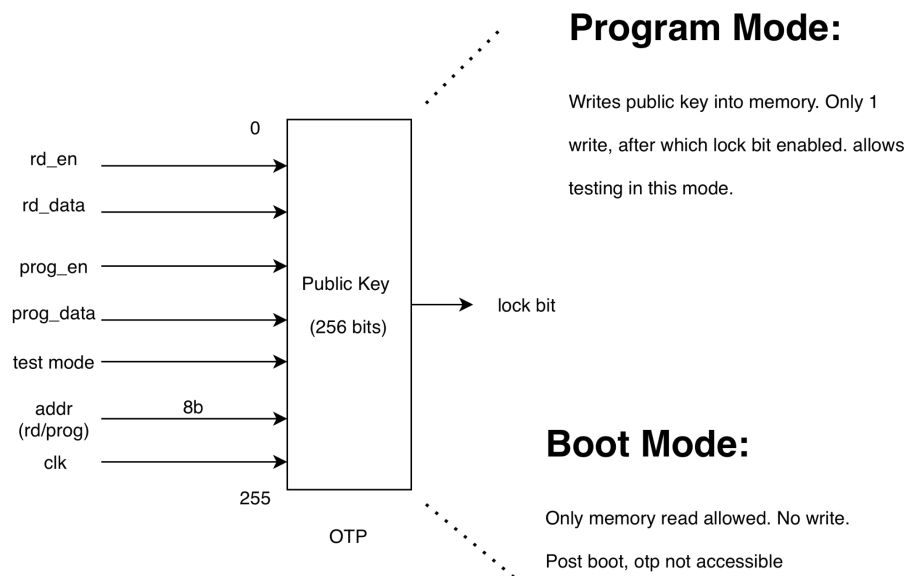
- **Capacity:** 1KB MaskedROM on-chip memory containing the primary bootloader code.
- **Architecture:** It is designed using a maskedROM macro as it is masked programmed, the contents are immutable after fabrication and characterization prevents any kind of tampering.
- The primary bootloader code stored contains the following instructions
 - Read firmware and signature (R, S) word by word from external flash via QSPI controller
 - Write msg_length and start command to buffer CSR
 - Write firmware words to buffer data port via OBI
 - Buffer streams firmware to SHA+ED25519 subsystem autonomously



- CPU polls signature_valid status to confirm verification result
 - If valid: copy verified firmware to ISRAM, set ISRAM lock bit in control registers, jump to ISRAM base (0x0001_0000)
 - If invalid: halt forever - ISRAM never written, execution never leaves BootROM
- BootROM forms a fundamental component of the hardware Root of Trust, ensuring that only authenticated firmware is executed.

One Time Programmable Memory (OTP):

- **Capacity:** 256-bit bit-addressable memory dedicated to a public key that is used for the secure boot verification, a part of the hardware Root of Trust.
- **Functionality:**
 - **Program Mode:** Supports a single write operation followed by a hardware Lock Bit to ensure the key is immutable after fabrication.
 - **Boot Mode:** Hardware-enforced read-only state. Post-boot logic ensures the OTP is completely inaccessible to application-level software.
- **Flow:** The public key is written into the OTP in program mode, once successfully written the lock bit goes high, indicating that the OTP is locked and no further data can be written into it. The secure boot process starts, where the OTP data can be read, but no write is allowed. Once the secure boot process is completed, the OTP is permanently locked with no further read or write possible.



Prefetch Buffer Implementation:

This section explains how the architecture maintains high throughput.

- **Specifications:** A 3 x 32-bit (96-bit total) buffer staged between instruction memory and the CPU decoder.
- **Functional Impact:**
 - **Latency Reduction:** Buffers instructions in advance to minimize Instruction Fetch (IF) latency.
 - **Pipeline Efficiency:** Ensures the execution pipeline remains saturated, preventing stalls during complex memory access cycles.

4. References

A. S. Siddiqui, G. Shirley, S. Bendre, G. Bhagwat, J. Plusquellic and F. Saqib, "Secure Design Flow of FPGA Based RISC-V Implementation," 2019 IEEE 4th International Verification and Security Workshop (IVSW), Rhodes, Greece, 2019, pp. 37-42, doi: 10.1109/IVSW.2019.8854418. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8854418&isnumber=8854381>

V. B. Y. Kumar, A. Chattopadhyay, J. Haj-Yahya and A. Mendelson, "ITUS: A Secure RISC-V System-on-Chip," 2019 32nd IEEE International System-on-Chip Conference (SOCC), Singapore, 2019, pp. 418-423, doi: 10.1109/SOCC46988.2019.1570564307. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9088035&isnumber=9087941>

R. Chaves, G. Kuzmanov, L. Sousa and S. Vassiliadis, "Cost-Efficient SHA Hardware Accelerators," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 16, no. 8, pp. 999-1008, Aug. 2008, doi: 10.1109/TVLSI.2008.2000450. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4560238&isnumber=4570466>

<https://github.com/lowRISC/ibex>
<https://ibex-core.readthedocs.io/en/latest/>

[EdDSA and Ed25519 | Practical Cryptography for Developers](#)

<https://datatracker.ietf.org/doc/html/rfc8032>

M. Bisheh-Niasar, R. Azarderakhsh and M. Mozaffari-Kermani, "Cryptographic Accelerators for Digital Signature Based on Ed25519," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 29, no. 7, pp. 1297-1305, July 2021, doi: 10.1109/TVLSI.2021.3077885.
<https://ieeexplore.ieee.org/abstract/document/9437467>

C. Palmiero, G. Di Guglielmo, L. Lavagno and L. P. Carloni, "Design and Implementation of a Dynamic Information Flow Tracking Architecture to Secure a RISC-V Core for IoT Applications," 2018 IEEE High Performance extreme Computing Conference (HPEC), Waltham, MA, USA, 2018, pp. 1-7, doi:10.1109/HPEC.2018.8547578.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8547578&isnumber=8547513>

<https://github.com/sld-columbia/riscv-dift>
<https://dl.acm.org/doi/abs/10.1145/3607199.3607246>

X. Deng, Y. Qiu, D. Toops and G. Jamison, "A Trim Bit One Time Programmable EPROM with Aggressively Reduced Area, Enhanced Functionality, and Extra Features," 2022 IEEE International Symposium on Circuits and Systems (ISCAS), Austin, TX, USA, 2022, pp. 1467-1471, doi: 10.1109/ISCAS48785.2022.9937313.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9937313&isnumber=9937203>

[One-Time Programmable \(OTP\) Memory Guide](#)

Wei Cui and Siliang Wu, "Design of Small Area and Low Power Consumption Mask ROM," 2007 IEEE International Conference on Integrated Circuit Design and Technology, Austin, TX, USA, 2007
<https://ieeexplore.ieee.org/document/4299585>